VERİ YAPILARI · 3. HAFTA

Bağlı Liste

singly linked list

GEÇEN HAFTADAN ÜÇ SORU

Önce kısa bir hatırlama yapalım

1. **Başa eklerken neden diğer elemanlar kaydı?**
2. **Python listesinde sona eklemek neden genelde hızlıydı?**
3. **class sözcüğü neyi tanımlar?**

Cevapları bugün kod üzerinde göreceğiz

Bağlı listeyi sıfırdan kuracağız.

GEÇEN HAFTANIN BIRAKTIĞI SORU

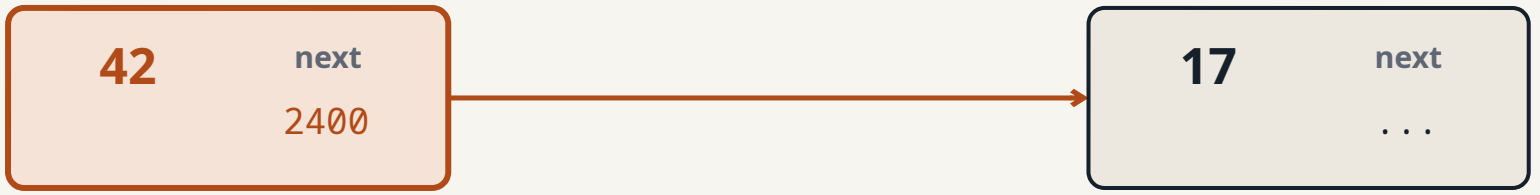
Başa eklerken kimseyi kaydırmasak olmaz mı?

Bugün bu soruya başka bir veri yapısıyla cevap vereceğiz.

TEMEL PARÇA

Düğüm nedir?

Düğüm (node), bir değeri ve bir sonraki düğümün adresini tutar.



değer

sonraki düğümün adresi

Her düğüm aynı iki bilgiyi taşır.

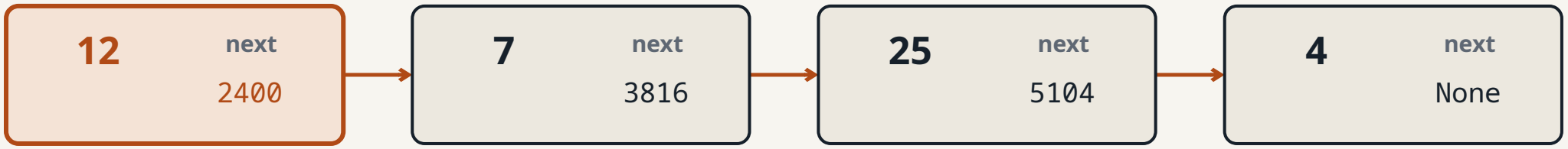
Düğümün bellekte nerede olduğu değerinden bağımsızdır.

SINGLY LINKED LIST

Tek yönlü bağlı liste

Düğümler, sonraki adreslerini göstererek bir sıra oluşturur.

head baş, listenin ilk düğümü



None listenin sonunu gösterir.

BUGÜNÜN ANA SORUSU

Kimseyi kaydırmadan başa ekleyebiliyorsak dizi neden hâlâ var?

Bir yapının kazancı başka bir işlemde bedel çıkarabilir.

PYTHON

class ile bir düğüm

class, aynı yapıya sahip nesnelerin tarifini yazar.

```
class Node:                          Düğümün (node) tarifi
    def __init__(self, data):        Yeni düğüm kurulunca çalışır
        self.data = data             Değeri düğümde saklar
        self.next = None             Sonraki düğüm henüz yok

first = Node(42)                     42 değerli bir düğüm kurar
print(first.data)                    Düğümdeki değeri yazdırır
```

Turuncu satır işlemin kaynağını gösterir

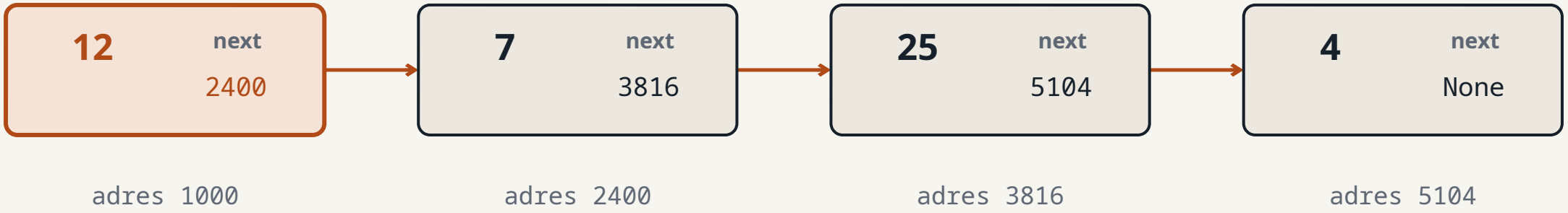
Düğümün iki alanı

data	veri, saklanan değer
next	sonraki düğüm

self nesnenin kendisini gösterir.

Düğüm bellekte tek kutu değildir

Her düğüm farklı bir adreste durabilir.



Dizideki gibi adres hesabı yapmayacağız. Bir sonraki adrese bakacağız.

Bağlantı adresi taşır

Listenin tamamına ulaşmak için ilk düğümün adresini bilmek yeterlidir.



head değişirse listenin ilk düğümü değişir.

Başlangıç adresini saklayan alana kodda head diyeceğiz.

Başa ekleme

Yeni düğüm listenin başına bağlanır.

1 **`new_node = Node(99)`**

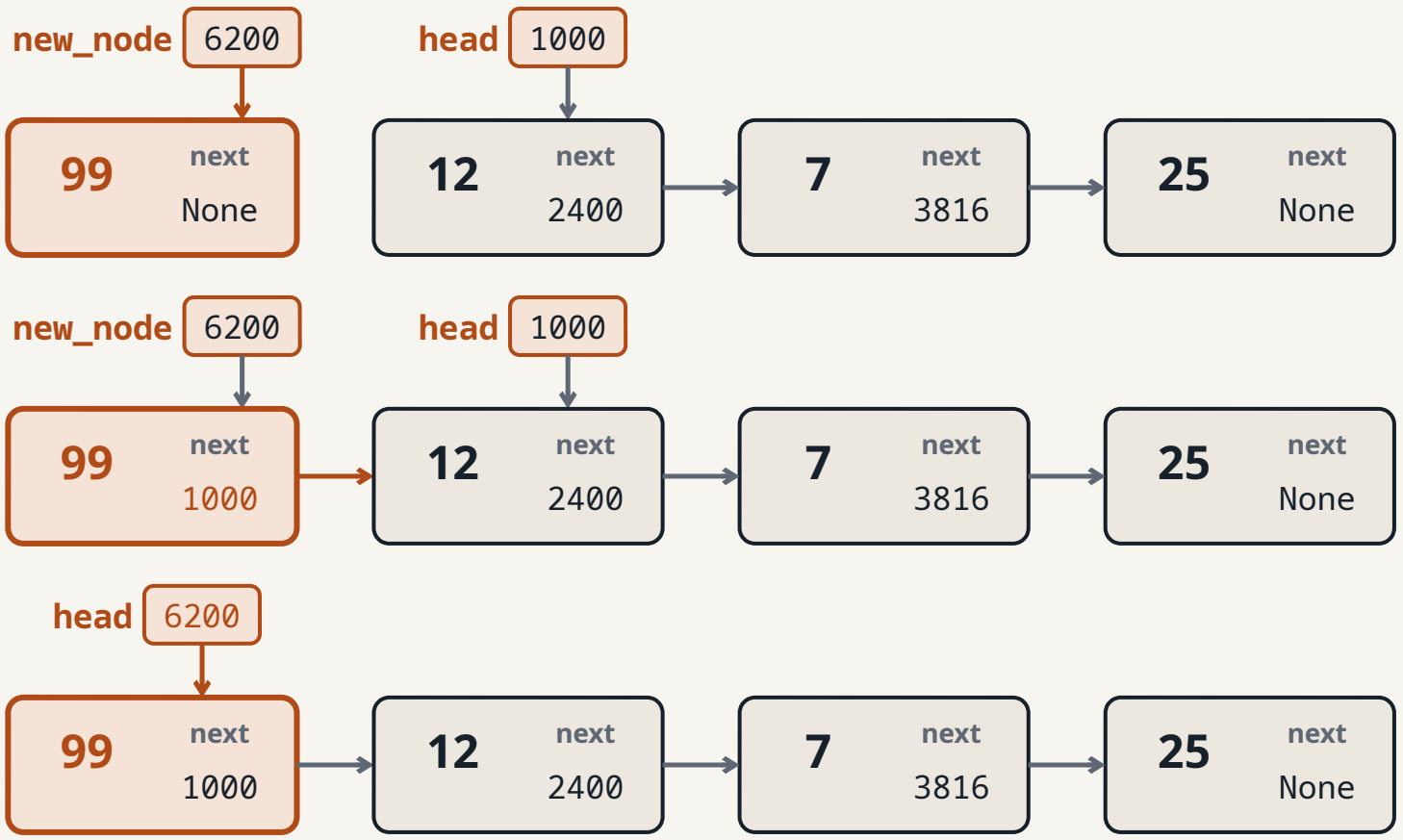
Yeni düğüm oluşur, next alanı None.

2 **`new_node.next = head`**

Yeni düğüm eski ilk düğümü gösterir.

3 **`head = new_node`**

head artık yeni düğümü gösterir.



Turuncu ok o adımda değişen bağlantıyı gösterir. 1000 ve 6200 gibi sayılar düğüm adresleri.

Başa ekleme kodla

prepend başa ekle, size boyut demek. Yeni düğüm, eski head değerini devralır.

```
def prepend(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
    self.size += 1
```

Liste boyundan bağımsız, O(1)

1. Yeni düğümü oluşturur
2. Yeni düğüm eski başı gösterir
3. Baş artık yeni düğüm

Eleman sayısı bir artar

Turuncu satır işlemin kaynağını gösterir

Üç bağlantı adımı

1. Düğümü oluştur
2. Eski başı göster
3. Başı güncelle

Başa eklemenin maliyeti

Listenin uzunluğu ne olursa olsun birkaç bağlantı değişir.

O(1)

sabit zaman

Veri 2 katına çıkınca

iş aynı kalır

Yeni düğüm yalnızca ilk düğümü gösterir.

Sona ekleme

Bu derste son düğümün adresini ayrıca saklamıyoruz.

head → 1 → 2 → 3 → None

Son düğümü bulmak için baştan başlayıp bağlantıları tek tek izleriz.

O(n)

Başa ekleme O(1) kalır. Sona ekleme için listeyi gezeriz.

Sona ekleme kodla

append sona ekle demek. Son düğümü bulmak için current ile dolaşırız.

```
def append(self, data):                 Sona ekleme, O(n)
    new_node = Node(data)               Yeni düğümü oluşturur
    if self.head is None:               Liste boş mu diye bakar
        self.head = new_node            Boşsa yeni düğüm baş olur
        self.size += 1                  Eleman sayısı bir artar
        return                          Boş liste için iş biter
    current = self.head                 current (gezici) baştan başlar
    while current.next is not None:     Son düğüme kadar döner
        current = current.next          n düğüm gezilir, O(n)
    current.next = new_node             Son düğüm yeniyi gösterir
    self.size += 1                      Eleman sayısı bir artar
```

Boş liste

head yeni düğümü gösterir

Dolu liste

current son düğüme ulaşır

Sona eklemenin maliyeti

Bu sınıfta yalnızca head var. Sona eklemek için son düğümü baştan buluruz.

Başa ekleme

head değişir

O(1)

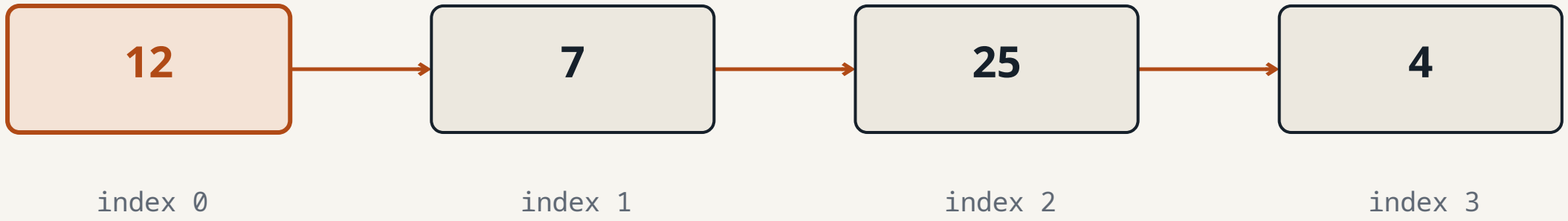
Sona ekleme

son düğümü bulur

O(n)

Sıra numarası yok, bağlantı var

Bağlı listede istenen sıra numarasına gitmek için bağlantıları izleriz.



Sıra numarasını bulmak için baştan ilerleriz.

500 numaralı sıraya gitmek

500 numaralı indekse ulaşmak için 500 bağlantıyı izleriz.



O(n)

500 numaralı indekse baştan 500 kez ilerleyerek ulaşırız.

Genel indeks erişimi en kötü O(n).

Arama

search ara, target aranan değer demek. Arama da düğümleri baştan sona kontrol eder.

```
def search(self, target):              Arama, en kötü O(n)
    current = self.head                current baştan başlar
    while current is not None:         Liste bitene kadar döner
        if current.data == target:     Aranan bu düğüm mü
            return current             Bulunca düğümü döndürür
        current = current.next         Sonraki düğüme geçer
    return None                        Bulamazsa None döner
```

Turuncu satır işlemin kaynağını gösterir

Aranan başta olabilir

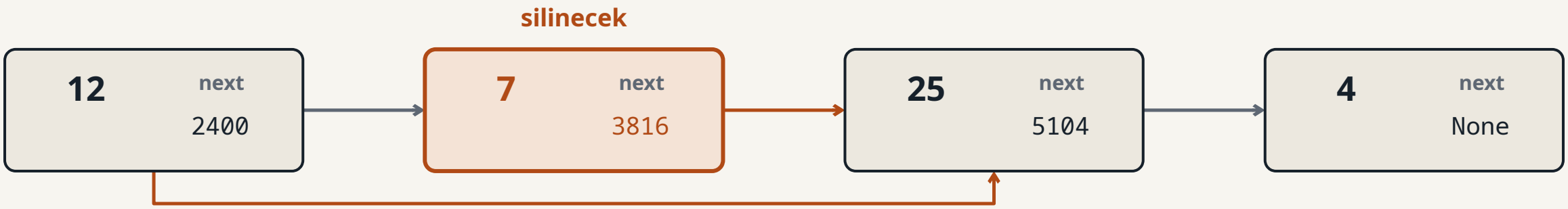
`en iyi durum O(1)`

Aranan sonda ya da yoksa

`en kötü durum O(n)`

Silme için önce öncekini buluruz

Ortadaki düğümü çıkarırken bağlantıyı üzerinden geçiririz.



12 → 25 `previous.next`

Silme kodla

delete sil demek. Düğümü bulurken bağlantıyı kaybetmemeliyiz.

```
def delete(self, target):                          Silme, en kötü O(n)
    if self.head is None: return False             1. durum, liste boşsa silinecek yok
    if self.head.data == target:                   2. durum, silinecek (target) ilk düğüm mü
        self.head = self.head.next                 Baş bir sonraki düğüme geçer
        self.size -= 1                             Eleman sayısı bir azalır
        return True                                Silme başarılı
    previous = self.head                           3. durum, previous (önceki) baştan başlar
    while previous.next is not None:               Sonraki düğüm varken döner
        if previous.next.data == target:           Sonraki düğüm silinecek mi
            previous.next = previous.next.next     previous, silinecekten sonrakini gösterir
            self.size -= 1                         Eleman sayısı bir azalır
            return True                            Silme başarılı
        previous = previous.next                   Bir düğüm ilerler
    return False                                   Değer listede yok
```

Boş liste ve tek eleman

Bağlantı güncellerken sınır durumlarını ayrıca düşünürüz.

Boş liste

```
head = None
size = 0
```

Tek eleman

```
head is not None
head.next = None
```

Tek eleman sil

```
head = None
size = 0
```

Bu kontroller kodun güvenli çalışmasını sağlar.

İşlem maliyetleri

Bu haftaki sınıfta yalnızca head ve size alanlarını tutuyoruz

İşlem	Bağlı liste	Neden
Başa ekleme	O(1)	head değişir
Sona ekleme	O(n)	sonu bulmak için gezer
Index ile okuma	O(n)	baştan ilerler
Arama	O(n)	bağlantıları izler
Silme	O(n)	öncekini bulur

Dizi mi bağlı liste mi?

Seçim, sık yaptığımız işlere göre değişir.

Dizi

index ile okuma O(1)

başa ekleme O(n)

referanslar bitişik alanda

Bağlı liste

başa ekleme O(1)

index ile okuma O(n)

düğümler farklı adreslerde

Hangi işlem sık yapılıyorsa ona göre seçeriz.

En sık yapılan hata

Yeni düğümü bağlamadan başı değiştirirsek listenin geri kalanını kaybederiz.

Yanlış sıra

1 **`head = new_node`**

head önce yeni düğüme taşınır.

2 **`new_node.next = head`**

Yeni düğüm kendini gösterir.



Doğru sıra

Önce `new_node.next = head`

Sonra `head = new_node`

Bağlantıyı koru, sonra başlangıcı değiştir.

ANA SORUNUN CEVABI

Başa ekleme ucuzladı.

Karşılığında sıra numarasıyla doğrudan erişim kayboldu.

Bağlı listeyi, sık yaptığımız işlemler başa ekleme ve bağlantı üzerinden ilerleme olduğunda seçeriz.

Kodda kullandığımız yapı

Notebook'ta düğüm ve listeyi iki sınıfla kuracağız.

```
class Node:                          Düğümün tarifi
    def __init__(self, data):        Düğüm kurulunca çalışır
        self.data = data             Değeri saklar
        self.next = None             Sonraki düğüm henüz yok

class LinkedList:                    Bağlı listenin (linked list) tarifi
    def __init__(self):              Boş liste kurulur
        self.head = None             Başta hiç düğüm yok
        self.size = 0                Eleman sayısı sıfır
```

Turuncu satır işlemin kaynağını gösterir

Ders sırası

1. Düğümü oluştur
2. prepend ve append
3. search ve delete
4. Çıktıyı birlikte oku

TAHMİN ALIŞTIRMASI

İlk üç fonksiyonun Big O değerini tahmin edin.

`get_first_value(linked_list)`	`O(  )`
`prepend(linked_list, value)`	`O(  )`
`append(linked_list, value)`	`O(  )`

Cevabı kağıda yazın.
Kod maliyeti söylemez.

İpucu

İşlem kaç düğüm gezebilir?

TAHMİN ALIŞTIRMASI

Son üç fonksiyonun Big O değerini tahmin edin.

```
search_value(linked_list, target)
```
O( )

```
get_value(linked_list, index)
```
O( )

```
delete_value(linked_list, target)
```
O( )

Cevapları notebook'ta
beraber kontrol edeceğiz.

Cevap için ipucu

İşlem kaç düğümü gezmek zorunda?

DERSİN TEK CÜMLESİ

Bağlı liste bağlantıları değiştirerek eklemeyi hızlandırır.

Bunun bedeli, sıraya göre erişimin yavaşlamasıdır.

GELECEK HAFTA

Geri al tuşu en son ne yaptığımızı nereden biliyor?

yığın · stack